

Industrial Project Report – Term 1

Topic: Large Language Model Building for Bankwise Processes

Author:

Wong Ko Yin

Academic Supervisor:

Prof. Irwin King

Industrial Supervisor:

Mr. Nathan Wong,

Mr. Edwin Chan

23 November 2024

Table of Contents

1. Introduction.....	3
1.1 Objectives	3
1.2 Overview of Risks and Challenges	3
1.2.1 Chinese Language Proficiency Risk.....	3
1.2.2 Data Privacy Risk	3
1.2.3 Hardware Challenges and Limitations	4
1.3 Scope of the Project.....	4
2. Solution Overview	4
2.1 Data Collection and Preparation.....	4
2.2 Model Selection and Initialization	5
2.2.1 Retriever	6
2.2.2 Large Language Models	6
2.3 Fine-Tuning Process	12
2.3.1 Hyperparameter Tuning	12
2.3.2 Training Environment and Resources	13
2.4 Evaluation Metrics and Procedures	13
2.4.1 Evaluation Metrics.....	13
2.4.2 Evaluation Procedures	16
3. Experiments and Results	19
3.1 Description of Experiments.....	19
3.2 Performance Metrics	24
3.3 Comparative Analysis.....	25
4. Challenges and Further Approaches	27
4.1 Challenges.....	27
4.2 Further Approaches	28
References	29

1. Introduction

1.1 Objectives

The goal for this project is to develop a chatbot system for Bank of Communication Hong Kong (BOCOM) to enhance the bank wise processes, especially in document searching and question-and-answer functionalities. By incorporating the Retrieval-Augmented Generation (RAG) techniques and Large Language Models (LLMs), the chatbot system is designed to facilitate the retrieval of information and regulatory requirements from a large corpus of internal documents.

1.2 Overview of Risks and Challenges

To develop the RAG chatbot system, there are several potential risks and challenges must be considered within the development process. These risks and challenges include Language Proficiency

1.2.1 Chinese Language Proficiency Risk

The first potential risk of this project is the about the Chinese Language of LLMs. Given that BOCOM is a Chinese bank, the Chinese proficiency of chatbot system is very important. According to Microsoft research, 88% of the world's languages lack access to LLMs because they are mainly trained on English data and for English speakers [1]. While LLM perform exceptional in English language proficiency nowadays, it often struggles with Chinese Language Tasks [2]. LLMs that have not been trained on Chinese datasets may struggle to understand the users' inquiries, interpret the internal documents and generate the accurate Chinese response. This limitation may have negative impact on the chatbot reliability in serving the Chinese bank.

Therefore, this project will only use the LLMs that have been trained on Chinese datasets and perform experiment to compare their capability in Chinese retrieval tasks. Evaluation of LLMs on multilingual tasks is important to ensure their effectiveness in real-world applications [3]. This experiment will ensure that the LLM used in the system has a great performance in Chinese retrieval task to meet the specific needs of BOCOM.

1.2.2 Data Privacy Risk

Another risk that should be considered is the data privacy risk for internal documents. Using RAG techniques requires to provide additional content to LLM, which may include confidential data and sensitive information. Using cloud machines or closed-source LLMs like Chat-GPT using API may raise confidentiality concerns because their confidential context may be disclosed to the service providers. When the chatbot system uses closed-source LLM or cloud computing service, the sensitive information included at the prompt and the internal documents could potentially be stored, accessed and used by unauthorized individuals.¹

Therefore, this project will download the open source LLM and deploy it using the local server as the foundation of the RAG system. This approach ensures chatbot system to run exclusively in the local machine to prevent unauthorized individuals from accessing the internal documents.

1.2.3 Hardware Challenges and Limitations

In addition to the above risks, the hardware challenge and limitations for this project also need to be given careful attention. Since this project requires running the open-source LLM in the local machine, a powerful GPU server is essential for the chatbot system's efficiency and effectiveness. The substantial Video random-access memory (VRAM) is significantly important for allowing inference of LLMs with large parameter sizes. For example, Llama 3.2 11B Vision requires at least 22 GB of VRAM, Llama 3.2 90B Vision: requires 180 GB of VRAM² and Qwen 2.5 72B requires 138 GB of VRAM³.

However, the GPU server provided by CUHK Fintech department has only at most 44 VRAM. This project can only use the smaller parameter size LLM. This challenge may limit the comprehensiveness of experiments on comparing different open-source LLMs on Chinese RAG tasks. Additionally, LLMs with the smaller parameter size may underperform compared to larger parameter size, which could affect the overall benchmarking and lead to less comprehensive conclusions of the experiments within this project.

¹ <https://www.security.org/digital-safety/is-chatgpt-safe/>

² <https://llamamodel.com/requirements-3-2/>

³ <https://huggingface.co/Qwen/Qwen2-72B/discussions/3>

1.3 Scope of the project

The scope of this project has the following key areas. It contains two experiments to select the most suitable LLM and RAG method. Once the model and method are identified, the next step is to fine-tune the selected model. After fine-tuning, a feedback loop will be integrated into the system. Finally, a prototype will be developed based on the fine-tuned model and most suitable RAG method.

1. **Comparison of open source LLM:** The project will involve a thorough comparison of various open-source LLMs to evaluate their capabilities in handling both Chinese and English RAG tasks. This will involve evaluating factors such as accuracy and effectiveness.
2. **Comparison of RAG Frameworks:** The project will involve a comparison of different RAG frameworks to identify the most suitable framework in integrating with the selected LLMs. This will involve evaluating factors such as accuracy, effectiveness and generation quality.
3. **Fine-tuning the LLM:** The project will include the fine-tuning approach to the selected LLMs to improve their performance in generating the response based on context retrieved. This process will focus on optimizing the models for accuracy and generation quality in RAG tasks.
4. **Human-in-the-Loop:** The project will involve a human-in-the-loop approach to ensure that human oversight is incorporated into the RAG system. This approach aims to improve the reliability of the chatbot's responses by allowing human input during interactions. It allows the system to handle various specific needs.
5. **Building the Prototype:** The project involves building a prototype of the RAG-based chatbot system. It would use the fine-tuned LLM and the most suitable RAG framework to showcase the system's functionality and capabilities.

2. Solution Overview

2.1 Data Collection and Preparation

The dataset used in this project is derived from the Hong Kong Monetary

Authority (HKMA) Supervisory Policy Manual, which includes the latest supervisory policies and approaches of the HKMA. These documents are in finance regulatory domain, which can simulate real internal documents used in BOCOM, providing a realistic foundation for testing and development of the RAG chatbot.

For the comparison of open-source LLMs and RAG frameworks, 7 Chinese and 2 English PDFs, selected by BOCOM, are used as the database for the RAG system. Since the BOCOM is a Chinese bank that most of documents are in Chinese language, these documents may simulate the typical use of Chinese materials supplemented by a smaller proportion of English content.

For the test set construction, 27 questions are generated from these 9 PDFs, with three questions per PDF. For the fine-tuning approach, 200 question-and-answer pairs with the relevant context will be generated as the training set.

2.2 Model Selection and Initialization

2.2.1 Retriever

This project uses Chuxin-Embedding⁴ model as the retriever to retrieve the relevant content of the user’s enquiries from the internal database. This embedding model is provided by Chuxin LLM team and designed to enhance the capabilities of Chinese text retrieval. Until 10 October 2024, it ranked the top of embedding model at Chinese retrieval task at Massive Text Embedding Benchmark (MTEB)⁵. Since this RAG chatbot system is mainly developed for Chinese documents, this model maybe the most suitable one for this project at this time.

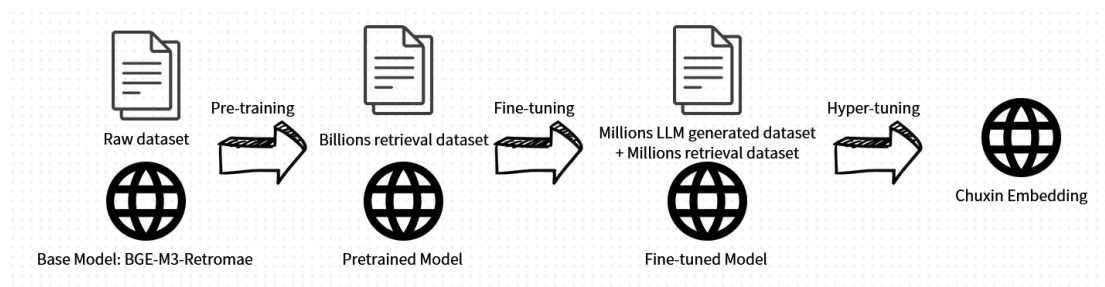


Figure 1: Training process of Chuxin Embedding

⁴ <https://huggingface.co/chuxin-llm/Chuxin-Embedding>

⁵ <https://huggingface.co/spaces/mteb/leaderboard>

Figure 1 shows the training process for the Chuxin-Embedding model. It is a multi-state approach including pre-training, fine-tuning and hyper-tuning. This model is pretrained on a billion-level raw dataset based on BCE-M3-Retromae architecture, which is a multilingual model including Chinese and English [3]. It is fine-tuned by a billion-level public retrieval dataset to enhance its retrieval capability. After fine-tuning process, Chuxin LLM used Qwen-72b, which is also a Chinese LLM trained by English and Chinese datasets [4], to generate a million level of retrieval datasets with a million-level public synthetic dataset to perform hyper-tuning⁶. This optimization step involves adjusting various hyperparameters of the model to further improve its performance on the retrieval task.

Table 1: Chuxin-Embedding Model Specifications
Chuxin-Embedding

Model Size:	568 million parameters ¹
Memory Usage:	2.12 GB ¹
Max Tokens	8194 tokens ¹
Embedding Dimensions	1024 ¹

Table 1 presents the specifications of Chuxin-Embedding model. It features 568 million parameters, and a memory usage of 2.12 GB. With the maximum token capacity of 8,194 tokens and embedding dimensions of 1,024, this model allows us to process a large size of datasets, making it particularly suitable for this chatbot system.

2.2.2 Large Language Models

This project aims to develop a robust RAG system handling both Chinese and English documents. To achieve this goal, 4 distinct series of LLMs are chosen to be evaluated, including two from US companies, Llama 3.2 and Gemma 2, as well as two from Chinese companies, Qwen 2.5 and Yi 1.5. Table 2 below shows the details of those LLMs.

Table 2: LLMs' Specifications

	Llama 3.2	Gemma 2	Qwen 2.5	Yi 1.5
Company:	Meta AI (US)	Google (US)	Alibaba Cloud (China)	01-AI (China)

⁶ <https://huggingface.co/chuxin-llm/Chuxin-Embedding>

Released Date	09/2024	06/2024	09/2024	05/2024
Model Size: (billion)	1b/3b/11b/90b	2b/9b/27b	0.5B/1.5B/3B/7B/14B/32B/72B	6b/9b/34b
Maximum Tokens	128,000	8192	3,200/128,000	128,000
Supported Language (Official)	English	English Chinese	English Chinese	English Chinese

Llama 3.2

Llama 3.2 is a series of LLMs within the llama community provided by Meta AI. According to table 2, this model series officially does not support Chinese. However, this model is trained by a large collection of languages which includes Chinese. It means that this model can still work on the Chinese documents, but the Chinese language is not the target language for Llama 3.2. The comparison of Llama 3.2 and other LLMs which officially support Chinese in this project will provide valuable insights into its performance of handling Chinese documents.

In addition, this model can handle at most 128,000 input tokens and it has variant model size, from 1 billion, 3 billion to 11 billion and 90 billion. 1B and 3B are its lightweight models, which provide the faster and more efficient options, while 11B and 90B are vision models, which provide powerful and complex options.

Lightweight Models (1B/3B) of Llama 3.2

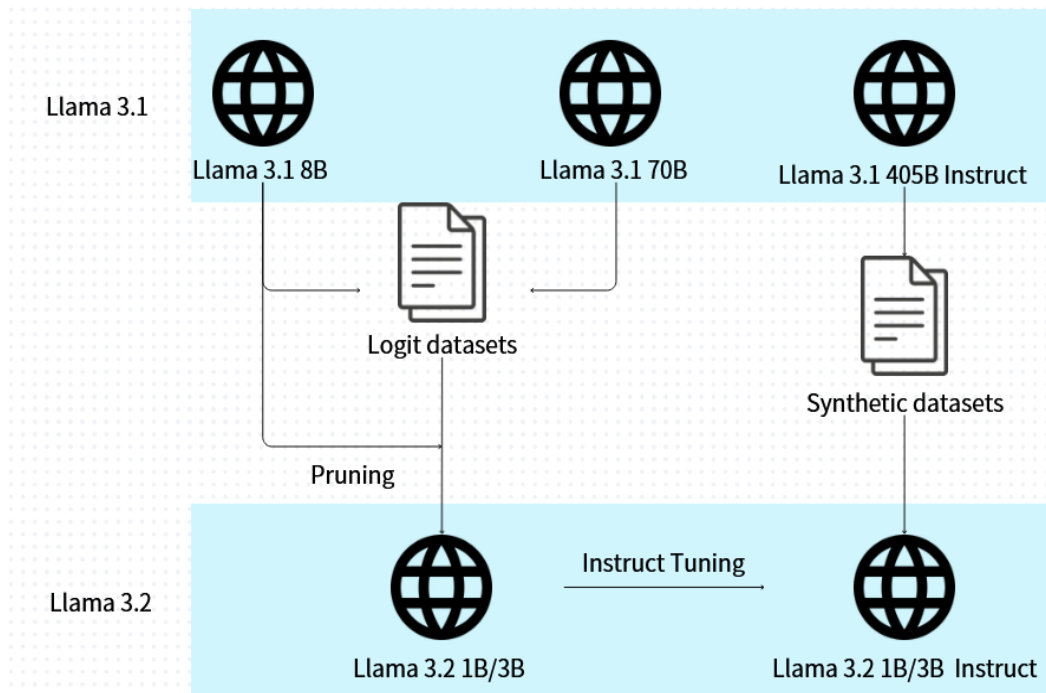


Figure 2: Model card for Llama 3.2's lightweight models (1B/3B)

Figure 2 shows the model architecture of base models and instruct models of Llama 3.2 1B and 3B. The construction of Llama 3.2 1B and 3B incorporated logits from Llama 3.1 8B and 70B. The 8B version is also used as teacher models to perform the knowledge distillation and structured pruning, maintain capability while improving the efficiency of Llama 3.2 1B and 3B.⁷ It is expected that the running time of lightweight models, 1B and 3B, will be notably speedy while maintaining acceptable accuracy.

As for instruct-tuned models for Llama 3.2 1B and 3B, the Llama 3.1 405B Instruct model is used to generate synthetic data for filtering and augmenting question and answers, providing high-quality instruction-tuning data. The data is used to perform instruction-tuning for Llama 3.2 1B and 3B, resulting in Llama 3.2 1B and 3B Instruct models. These instruction-tuned models are expected to have better ability to understand and respond to user queries effectively.

Vision Models (11B/90B) of Llama 3.2

⁷ <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>

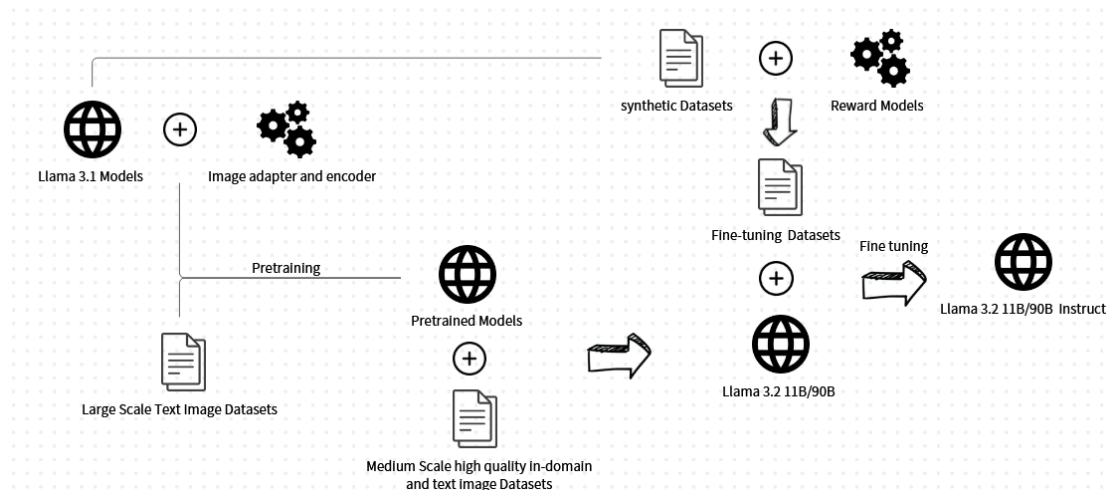


Figure 3: Model card for Llama 3.2's vision models (11B/90B)

Figure 3 shows the model architecture of base models and instruct models of Llama 3.2 11B and 90B. Similar to the lightweight models, the construction of Llama 3.2 11B and 90B are based on Llama 3.1 models. Other than the lightweight models, these models are vision models that are expected to handle both image and text input. Therefore, they are pretrained with the image adapter and encoder and the large-scale text-image datasets. Then, the medium scale high quality in-domain and text-image datasets are used to enhance its domain-knowledge capabilities. It is expected that the running time of vision models, 11B and 90B, will be longer than the lightweight models, but they would have better accuracy.

As for instruct-tuned models for them, similar to the lightweight models, the Llama 3.1 model is used to generate synthetic data for filtering and augmenting question and answers, providing instruction-tuning data. The main difference is that they also used the reward model to rank all the candidate answers that provide higher quality data. The data is used to perform instruction-tuning for Llama 3.2 11B and 90B, resulting in Llama 3.2 11B and 90B Instruct models.

Gemma 2

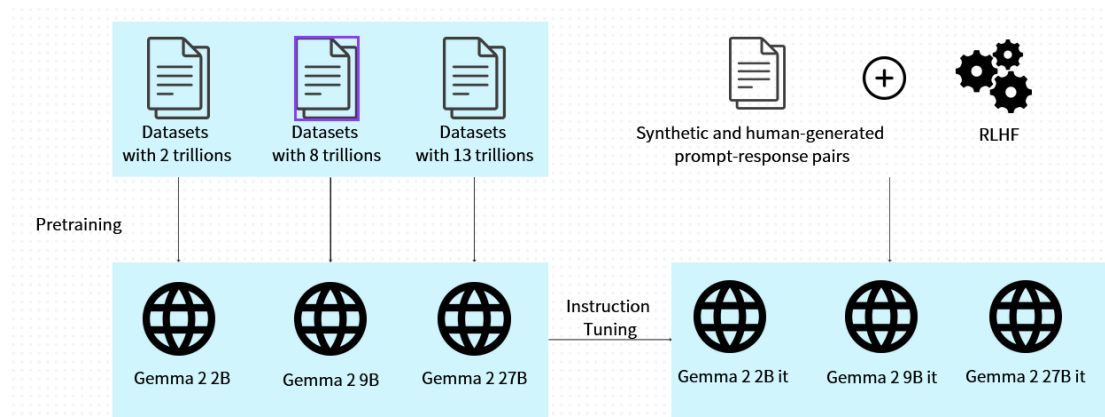


Figure 4: Model card for Gemma 2 models

According to Table 2, Gemma 2, 2B, 9B, and 27B were developed by Google and support both Chinese and English. They can handle a maximum of 8,192 tokens, which is lower than that of the other three LLMs. This may present a disadvantage, as the relevant content provided to the LLMs may be limited.

Figure 4 shows the model architecture for Gemma 2. Different parameter sizes of the models are trained on varying dataset sizes, ranging from 2 trillion to 13 trillion tokens. After that, the Reinforcement Learning from Human Feedback (RLHF) algorithm is applied as a reward model to the synthetic and human-generated prompt-response pair datasets to perform instruction tuning for the base models of Gemma 2, resulting in the instruction-tuned models of Gemma. [5]

Qwen 2.5

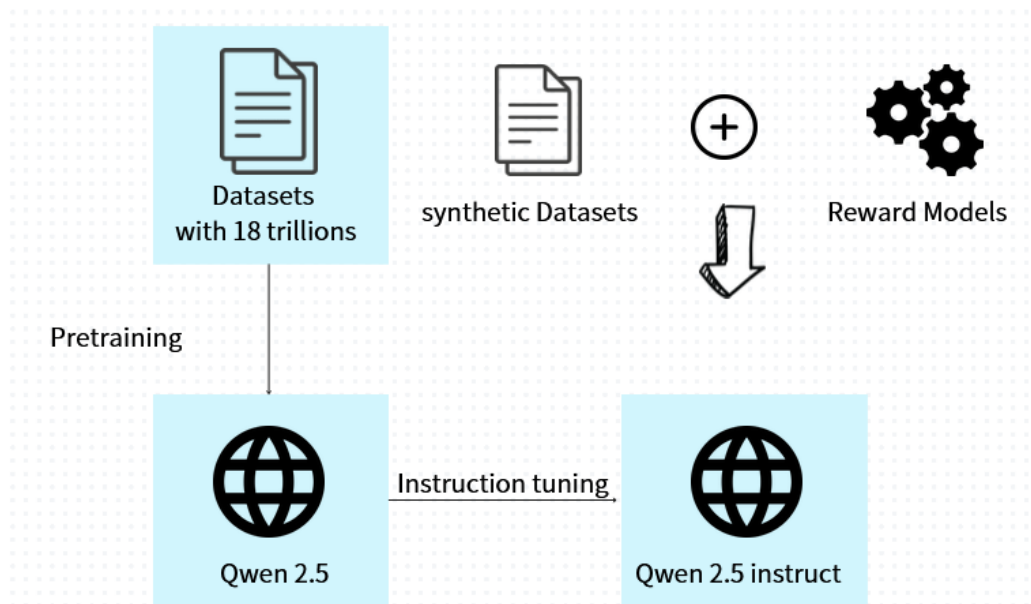


Figure 5: Model card for Qwen 2.5

Qwen 2.5 models are developed by Alibaba Cloud, and this version is an extension of the Qwen 2 models. They support over 29 languages, including Chinese and English. The maximum input tokens for parameter sizes of 0.5B, 1B, and 3B are 3,200, while for parameter sizes of 7B, 14B, 32B, and 72B, the maximum is 128,000. Unlike the above two LLMs, the maximum input tokens of Qwen 2.5 depend on its model size. A larger model must be chosen if more relevant content needs to be included in the prompt.

Figure 5 shows the model architecture for Qwen 2.5. Apart from Gemma 2, all Qwen 2.5 models are pretrained on 18 trillion datasets, meaning that the existing knowledge across different model sizes of Qwen 2.5 is similar. However, the instruction tuning approach is comparable to that of the above two LLMs; they utilize synthetic datasets with a reward model to produce fine-tuning datasets for the instruction tuning of the base model.

Yi 1.5

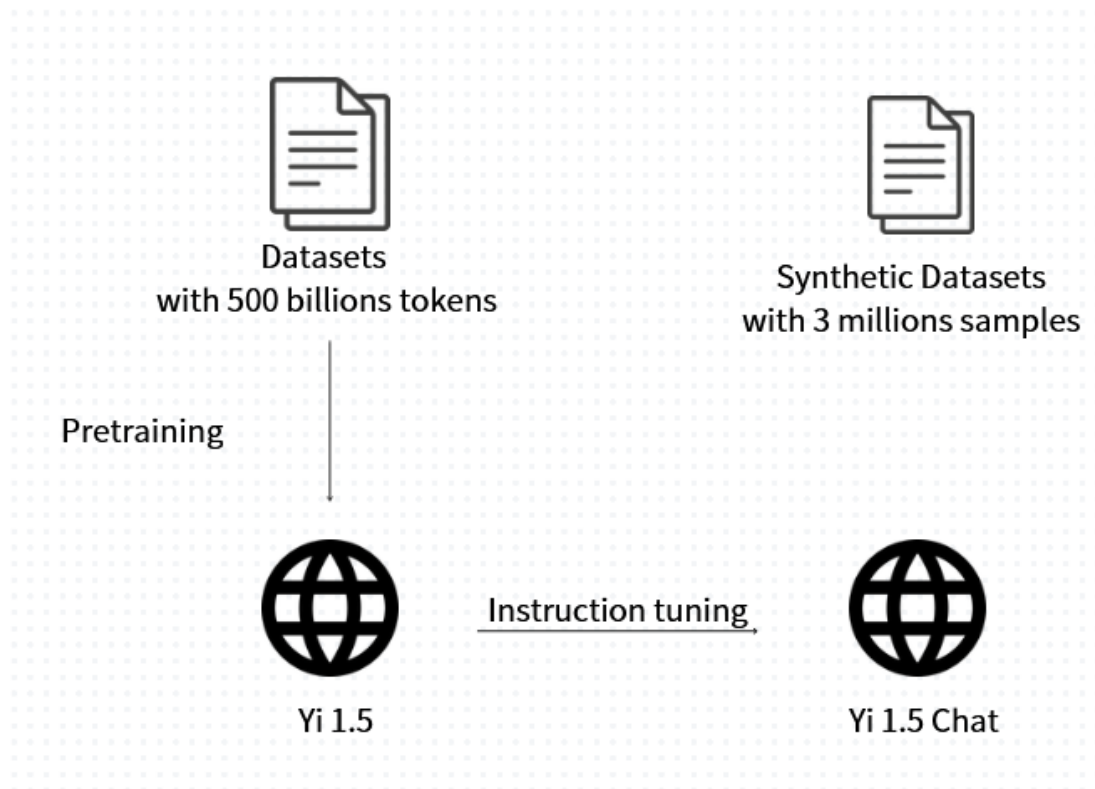


Figure 6: Model Card for Yi 1.5

Yi 1.5 is developed by 01 AI and released on 05/2024. It has 3 different model sizes, 6B, 9B and 27B. They officially support Chinese and English languages and can handle up to 128,000 input tokens, which is similar to Llama 3.2 and the larger model of Qwen.

The base models of Yi 1.5 are pretrained on web datasets with 500 billion tokens and the instruction-tuned models are tuned by synthetic datasets with 3 million samples, which includes prompt-response pair to enhance the models' capabilities on respond users' queries.

2.3 Fine-tuning Process

2.3.1 Hyperparameter Tuning

This project would compare the performance of those 4 LLMs on RAG tasks and choose the best one to perform the fine-tuning process to enhance to model's RAG capabilities. From the HKMA Supervisory Policy Manual mentioned above, 200 question-and-answer pairs would be generated to perform the fine-tuning approach.

The fine-tuning data would apply in the same format as Stanford Question Answering Dataset (SQUAD) [8, 9, 10, 11]. Each pair would have question, retrieved context and the suggested answer. Retrieval Augmented Fine-Tuning (RAFT) [12,13,14] method would be applied to improve the interplay between retrieval and generation of the LLM.

2.3.2 Training Environment and Resources

Table 3: Details of GPU Resources

GPU card	4 * Nvidia 1080 TI
GPU RAM	44 GB
Motherboard RAM	916 GB

The training environment plays a crucial role in efficient fine-tuning. To achieve optimal performance, this project uses the Graphics Processing Units (GPUs) server provided by the CUHK Fintech Department. The server contains four Nvidia 1080 TI cards, each with 11 GB of CUDA memory, for a total of 44 GB.

Due to the constraints of CUDA memory, only a limited range of LLM model sizes can be utilized in this project. The models available include:

- Llama 3.2: 1B, 3B, 11B
- Gemma 2: 2B, 9B
- Qwen 2.5: 0.5B, 1.5B, 3B, 7B
- Yi 1.5: 6B

2.4 Evaluation Metrics and Procedures

2.4.1 Evaluation Metrics

Accuracy

To evaluate the performance of the RAG system, accuracy is the most important aspect as we need to determine whether the system can generate our target response. Papers [19, 20] suggest several evaluation metrics, including BLEU, ROUGE, and METEOR, to measure the accuracy of RAG models. This project will choose the **METEOR** (Metric for Evaluation of Translation with Explicit Ordering) method to calculate accuracy. The reason for choosing this method is that it incorporates both precision and recall, whereas BLEU focuses only on precision and ROUGE focuses

only on recall [21]. Additionally, according to papers [22, 23], the METEOR method has a better ability to capture semantic similarities and reflect sentence-level quality.

The score is scaled to the range of (0, 1), where 1 indicates that the model's response completely matches the ground truth, and lower values represent how far the model's response is from the ground truth.

1. METEOR Score (Model Response $\leftrightarrow$ Target Response)

$$P = \frac{m}{w_t}, R = \frac{m}{w_r}, F_{mean} = \frac{10PR}{R + 9P}$$

$$Penalty = 0.5 \left(\frac{C}{U_m} \right)^3, METEOR = F_{mean}(1 - Penalty)$$

where:

m: Number of unigrams in the model response also found in target response

w_t : Number of unigrams in the model response

w_r : Number of unigrams in target response

C: Number of chunks in the model response

U_m : Unigrams in the model response

Equation 1: Formula of METEOR Score

Generation Quality

In addition to accuracy, the generation quality is also important. This evaluation method is generally used for evaluating LLM at RAG tasks. This aspect would adopt the evaluation strategies proposed by Retrieval Augmented Generation Assessment (RAGAs) [17]. There are 2 strategies to access the generation quality of RAG models, including Faithfulness and Answer Relevance.

Faithfulness measures factual consistency of the generated answer against the given context [18]. This is important for preventing hallucinations and ensuring that the retrieved context serves as a justification for the generated answer [17]. GPT-3.5-Turbo from Azure OpenAI is used as the LLM to determine whether the claim is consistent with the retrieved context. The reason for choosing Azure OpenAI is that it is supported by the RAGAs library and provides free credits for students to access GPT-3.5-Turbo.

Table 4: Prompt Template for using LLM to judge faithfulness

Instruction	Your task is to judge the faithfulness of a series of statements based on a given context. For each statement you must return verdict as 1 if the statement can be directly inferred based on the context or 0 if the statement can not be directly inferred based on the context.
Example Input	Context: "....." Statement: [".....", ".....", ".....", "....."]
Example Output	Statement: "....." Reason: "....." Verdict: (0/1) (Repeat many times)
My Input	Context: "....." Statement: [".....", ".....", ".....", "....."]

Table 4 is the prompt template proposed by RAGAs tools that use LLM to judge faithfulness. The LLM will output the verdict of different statements, either 0 or 1, to judge whether the claims in model response can be inferred based on the provided context. The faithfulness score is calculated by the percentage of number of claims in model response that are consistent with the provided context.

The score is scaled to the range of (0, 1), where 1 indicates that all claims in LLM's response can be inferred from the given context, and lower values represent how inconsistent the model's response is from the given context.

2. Faithfulness (Retrieved Context ⇔ LLM Response)

$$Faithfulness = \frac{C}{T}$$

where:

C is Number of claims in model's response that are consistent with the retrieved context,

T is Total number of claims in model response

Equation 2: Formula of Faithfulness Score

Answer relevance calculates the cosine similarity between the embeddings of user query and the model responses. The embeddings model would use the same model as the retriever: Chuxin-Embedding.

The score is scaled to the range of (0, 1), where 1 indicates the model response better can address the user query, and lower values represent that the model response is incomplete or contain redundant information.

3. Answer Relevance (User Query $\leftrightarrow$ LLM Response)

$$\text{Answer Relevance} = \frac{1}{n} \sum_{i=1}^n \text{sim}(q, q_i)$$

Equation 3: Formula of Answer Relevance Score

Efficiency

In addition, the running time for each model will also be taken into account, as excessive execution times may discourage users from using this system.

2.4.2 Evaluation Procedures

1. Database Development

Table 5: Database

Topic	Language	Size (kb)	Tokens
Corporate Governance of Locally Registered Authorized Institutions	Chinese	771.7	38912
Climate risk management	English	490.1	35840
Guidelines on combating money laundering and terrorist financing (for authorized institutions)	Chinese	1100	84992
Risk-based regulatory regime	Chinese	672.0	30720
Outsourcing	Chinese	173.9	14336
Regulatory approach to combating money laundering and terrorist financing	Chinese	708.9	20480
Risk management in electronic banking	English	337.0	37888
General Principles of Technology Risk Management	Chinese	261.9	29696
Ongoing business operations planning	Chinese	551.3	24576
Total	7 Chinese 2 English	5066.8	317440

The database consists of 9 PDF documents sourced from the Hong Kong Monetary Authority (HKMA) Supervisory Policy Manual⁸ as shown at table 5. It reflects the multilingual nature of the domain, including 7 Chinese documents are in Chinese and 2 English documents. This distribution simulates the typical document

⁸ <https://www.hkma.gov.hk/chi/regulatory-resources/regulatory-guides/supervisory-policy-manual/>

composition within bank of Communication, where most documents are expected to be in Chinese, while a smaller proportion in English. The memory usage of the database is 5066.8 kb, where contains 317440 tokens. The token count is calculated by the retriever used in this project, Chuxin embedding.

The langchain community vector database function was applied to store the text extracted from the PDFs [24, 25, 26]. The Chuxin embedding model was used as the retriever to encode the text and retrieve the relevant context during the RAG process.

2. Test Set Construction

A total of 27 question-and-answer (Q&A) pairs were generated as the test set to evaluate the models, where 3 questions were created for each of the 9 PDFs, ensuring comprehensive coverage of the database content. The questions were carefully designed to reflect the practical and domain-specific queries that users might ask based on the content of the HKMA Supervisory Policy Manual. Answers were generated by humans to establish a reliable ground truth for evaluation.

3. Experiments

3.1 Comparison of Open-Source LLMs

This step evaluates and compares the performance of selected open-source LLMs to identify the top three models for further experimentation. Due to the limitation of GPU RAME, only models mentioned at Part 2.4.2 are used in this experiment.

The evaluation is conducted under two conditions: baseline (without retrieval) and RAG (retrieval-augmented generation). To ensure fair comparisons, identical prompts are used for all LLMs under the same condition and the same retrieved context is provided for RAG-based evaluation.

- **Baseline (No-RAG Method):** Under this condition, the prompts are directly provided with the questions from the test set without any additional context. Model responses are generated based solely on the LLMs' pre-trained knowledge.
- **RAG Method:** In this condition, the LangChain community vector database is used to retrieve the most relevant context from the database using the Chuxin embedding model. The retrieved context is provided to the prompts along with the question. This method enables LLMs to generate responses based on the

content provided from the database.

3.2 Comparison of Different RAG Frameworks

The top three LLMs selected for the first experiment will be used for this phase. This experiment is to research and implement alternative RAG frameworks, using the same database and test set are used for consistency. These methods aim to optimize retrieval and contextual integration to enhance the overall performance of the system.

4. Fine-tuning Approach

Dataset Splitting:

The SQUAD dataset of 200 question-answer pairs is split into the training set and test set.

- **Training Set (80%):** Used for fine-tuning the model using the Retrieval-Augmented Fine-Tuning (RAFT) approach.
- **Validation Set (20%):** Used for hyperparameter tuning and monitoring the fine-tuning process to prevent overfitting.
- **Test Set (27 Q&A pairs):** The same test set as the 2 experiments will be used to ensure consistency when comparing the performance of different models.

Baseline Comparison:

The original LLM will be used as the baseline model to evaluate whether the fine-tuned LLM makes improvement. METEOR score, faithfulness, and answer relevance metrics will be used as evaluation metrics to evaluate the accuracy and the generation quality.

Error Analysis:

The error analysis is to analysis all the cases that has lower accuracy and identify the reasons of this, including the retrieval quality and generation quality. The error analysis would provide insights to further improvements.

3. Experiments and Results

3.1 Description of Experiments

This project contains two main comparisons. The first is to compare the performance of different open-source LLMs in Chinese RAG tasks. The second is to compare different RAG frameworks. The first experiment would evaluate all available LLMs, while the second experiment only uses the top 3 models during the first experiment.

1. Comparison of open source LLM

In comparing the different open-source LLMs on Chinese RAG tasks, this experiment would adopt 3 aspects mentioned above, accuracy, generation quality and running time. LLM without using RAG method would also be used as the baseline to determine whether the RAG method improves the accuracy of the chatbot system and which LLM can provide the target response.

Table 6: Prompt template for comparison of different LLMs on RAG tasks

Prompt Template	
No RAG (Baseline)	RAG
使用不超過 300 字繁體中文(連標點) 來回答問題。 Question: {question} Answer: ""	使用不超過 300 字繁體中文(連標點) 回答所給的上下文來回答問題。 上下文: {Relevant context} Question: {question} Answer: ""

Table 6 shows the prompt template for this experiment. The prompts use Chinese language as this project is focus on the Chinese RAG tasks. The difference between the two prompts is that one includes relevant context, while the other does not. By excluding RAG as a baseline, we aim to determine whether incorporating relevant context through the RAG method enhances the accuracy of the responses generated by the model.

Additionally, this experiment will assess the performance of different open-source LLMs mentioned in Part 2.2 in RAG tasks based on three key metrics: accuracy, generation quality, and running time. This comparison allows us to identify the most suitable LLMs for our project.

2. Comparison of RAG frameworks

After selecting the most suitable LLMs, the next phase involves researching and experimenting with various RAG methods. The goal of this phase is to explore

innovative approaches that can further enhance the accuracy of the responses generated by the chosen LLM. This experiment will evaluate different techniques for retrieving relevant context and integrating it into the generation process, with the aim of improving the accuracy of our RAG system. Unlike the first experiment, the evaluation will focus on accuracy and running time, excluding generation quality. It is because the emphasis of generation quality is on assessing the performance of the LLMs, but this experiment primarily targets research into different RAG frameworks to enhance accuracy.

RAG

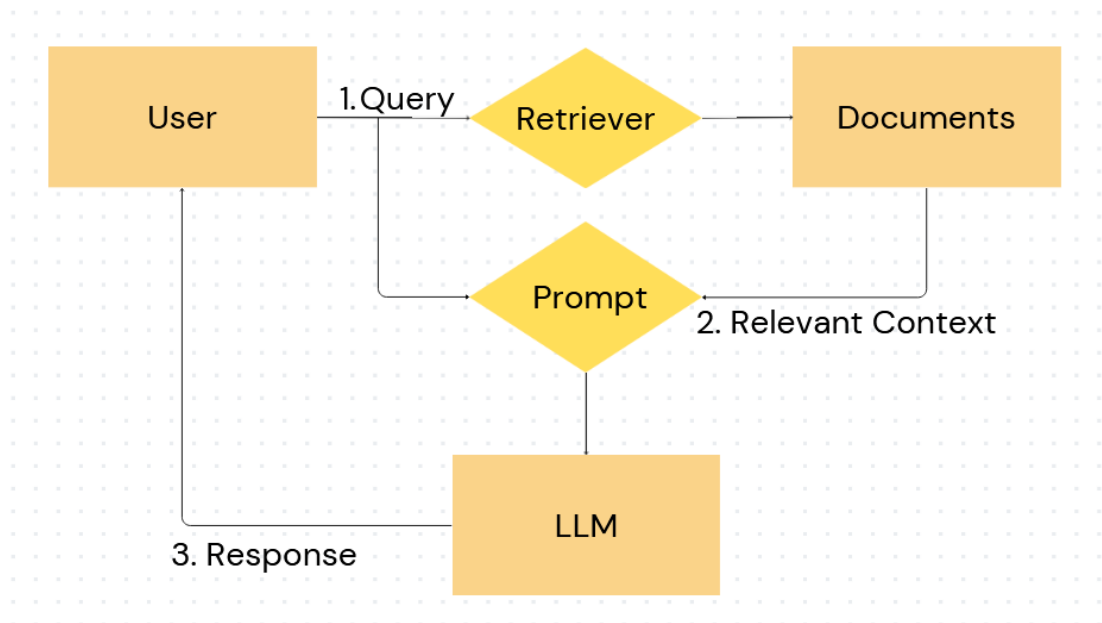


Figure 7: Basic RAG Method

Figure 7 is the architecture for the basic RAG method. The process starts with the user's query. The retriever then retrieves relevant documents from the database. Then, the query with the retrieved context is used to generate the prompt based on the prompt template in table 6. After that, this prompt is input into the language model (LLM), which generates a response that is returned to the user.

Self-Reflective RAG

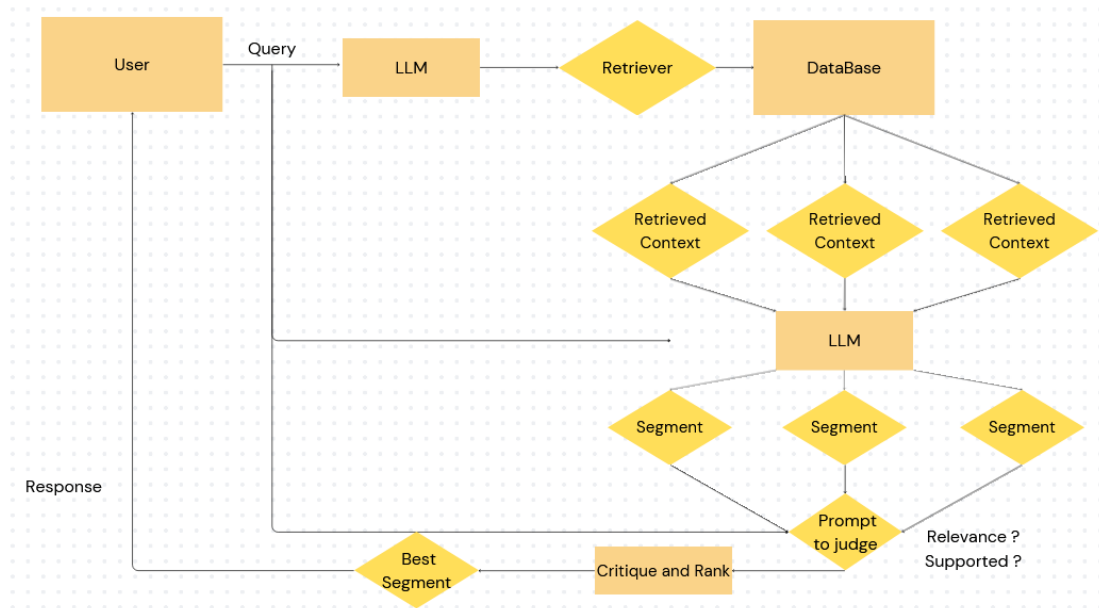


Figure 8: Self-Reflective RAG

Self-Reflective RAG [27] is an advanced framework that enhances basic RAG by incorporating self-reflection and critique into the generation process. It allows the model to iteratively refine its outputs by criticizing them to ensure relevance, and factuality.

Figure 8 shows the step-by-step approach of self-Reflective RAG. Similar to basic RAG, the process starts with the user query and top-k relevant context from database. Then, the LLM generates responses for user’s query based on each relevant context. Each response is referred to as a segment. After that, the LLM is employed to rank and critique whether the segment is relevance to the user’s query or supported by the relevant document. The best segment would be used as the final output of the system. Below is the prompt template for LLM to judge whether the segment is relevant or supported.

Table 7: Prompt Template for Critique

Prompt to judge	
Instruction:	You are tasked with evaluating the quality of a response segment based on its relevance and supportiveness with respect to the user query and retrieved document. ISUSE: Assess whether the retrieved document was used in

	generating the response. This means the response relied on the document to provide its content. Answer with "Yes" or "No". ISSUP: Assess whether the response segment is supported by the retrieved document. A response is supported if it directly uses information from the document or aligns with its content. Answer with "yes" or "no".
Example	User Query: What is the capital of France? Retrieved Context: Paris is the capital city of France and one of its major centers of commerce and culture. Generated Response Segment: The capital of France is Paris. ISSUP: Yes ISUSE: Yes
Input	User Query: {query} Retrieved Context: {Relevant Context} Generated Response Segment: {response} ISSUP: ISUSE: *3

Example: Llama-3.2-11B-Instruct

Table 8: Example for using Self-reflective RAG

User Query	電傳轉帳是甚麼?				
Relevant Context 1	Relevant Context 2		Relevant Context 3		
電傳轉帳是由一間機構(匯款機構)代表某人(匯款人)藉電子方式進行的交易，目的是將某筆金錢轉往某間機構(收款機構)(該機構可以是匯款機構或另一機構)以提供予該人或另一人(收款人)，而無論是否有一間或多於一間其他機構(中介機構)參與完成有關金錢轉帳。	認可機構 應就其在電傳轉帳中扮演的角色，遵從本章所載的相關規定。		如認可機構是電傳轉帳的匯款人或收款人，即並非以匯款機構、中介機構或收款機構的身分行事，則無需就該交易遵守附表 2 第 12 條的規定或本章的指引。		
Model Response 1	Model Response 2		Model Response 3		
電傳轉帳是由一間機構(匯款機構)代表某人(匯款人)藉電子方式進行的交易，目的是將某筆金錢轉往另一間機構(收款機構)以提供予該人或另一人(收款人)，而無論是否有一間或多於一間其他機構(中介機構)參與完成有關金錢轉帳。	電傳轉帳是把資金從一個國家的銀行帳戶轉到另一個國家的銀行帳戶。這種方式通常透過 SWIFT 系統進行，具備快速、安全的特點，但可能會涉及手續費。		如果認可機構是以電傳轉帳的匯款人或收款人的身分進行交易，而非作為匯款機構、中介機構或收款機構的角色，則該機構無需遵守附表 2 第 12 條的規定或相關指引。		
Supported?	Relevant?	Supported?	Relevant?	Supported?	Relevant?
Yes	Yes	No	Yes	Yes	No
Target Response	電傳轉帳是由一間機構代表某人藉電子方式進行的交易，目的是將某筆金錢轉往某間機構以提供予該人或另一人，而無論是否有一間或多於一間其他機構參與完成有關金錢轉帳。				Accuracy (METEOR)
Self-Reflective RAG	電傳轉帳是由一間機構(匯款機構)代表某人(匯款人)藉電子方式進行的交易，目的是將某筆金錢轉往另一間機構(收款機構)以提供予該人或另一人(收款人)，而無論是否有一間或多於一間其他機構(中介機構)參與完成有關金錢轉帳。				0.8783

Compared to standard RAG	電傳轉帳是由匯款機構代表匯款人以電子方式進行的交易，將資金從一個國家的銀行帳戶轉至另一個國家的銀行帳戶，通常透過 SWIFT 系統完成，具備快速和安全的特點，但可能涉及手續費。如果認可機構是以匯款人或收款人的身分進行電傳轉帳，而非作為匯款機構、中介機構或收款機構，則無需遵守附表 2 第 12 條的規定或相關指引。整個過程可包括一間或多間中介機構協助完成資金轉移。	0.2877
--------------------------	--	--------

This example shows how the self-reflective RAG work and how it gets improvement from the standard RAG. The LLM used in this example is Llama-3.2-11B-Instruct and the retriever is the chuxin embedding.

It started with retrieving the top 3 relevant context of user's query from database. The standard RAG method used all relevant contexts to generate a single response, while the self-reflective RAG separated 3 relevant contexts and generated the response for each context individually. After generating 3 responses, it allowed LLM to judge themselves whether their response is supported by the relevant document or relevant to the user query. The target response is the best response under these 2 conditions.

The performance of the example using the METEOR metric, which measures the similarity between the generated response and the target response. The Self-Reflective RAG model achieves a high accuracy score of 0.8783, indicating that its response closely aligns with the target response. In contrast, the standard RAG model achieves a much lower accuracy score of 0.2877.

3.1 Performance Metrics

Metrics used in comparison of different LLMs

- **Accuracy (METEOR):** It measures how correctly the LLMs answer the 27 test set questions compared to the ground truth answers. A higher accuracy score indicates that the model response is similar to the expected answers. The score can be used to determine which LLMs provide the most accurate responses both when relying on pre-trained knowledge (Baseline) and when augmented with retrieved context (RAG).
- **Efficiency (Response Time):**
The running time measures the duration of the model response to the user query. A low running time means the model can quickly answer the user query.

Comparison of different RAG frameworks

- **Accuracy (METEOR):** This measures how accurately the RAG frameworks enable the LLM to answer the user query correctly. It is used to compare the RAG framework's ability to support accurate response.
- **Generation Quality:** This metric evaluates the faithfulness and answer relevance to compare how well different RAG frameworks leverage retrieved context to produce high-quality, focused answers. Frameworks that improve faithfulness and relevance are considered more effective.
- **Efficiency:** This metric measures the running time of each RAG framework when integrated with the LLMs. The faster frameworks would be more favorable in real world applications.

3.2 Comparative Analysis

Comparison of different LLMs

Table 9: Experiment Result for Comparison of LLMs

Metrics	Accuracy		Response Time		Output Tokens	
Method	No RAG	RAG	No RAG	RAG	No RAG	RAG
Llama-3.2-1B	0.0672	0.2334	1.81s	6.41s	55	80
Llama-3.2-1B Instruct	0.1270	0.3251	2.19s	6.07s	96	97
Llama-3.2-3B	0.0699	0.2456	6.85s	12.74s	86	142
Llama-3.2-3B Instruct	0.1332	0.4693	10.04s	13.81s	187	182
Llama-3.2-11B	0.1616	0.4362	11.67s	24.89s	200	200
Llama-3.2-11B Instruct	0.1949	0.4984	10.52s	17.67s	173	111
Gemma-2-2B	0.0878	0.2080	13.22s	15.07s	110	195
Gemma-2-2B it	0.1393	0.4081	6.04s	7.56s	131	126
Gemma-2-9B	0.1233	0.2363	21.96s	30.00s	138	180
Gemma-2-9B it	0.1816	0.5045	11.89s	16.37s	99	70
Qwen-2.5-1.5B	0.1401	0.3557	5.74s	6.44s	91	94
Qwen-2.5-1.5B Instruct	0.1721	0.3588	2.19s	6.44s	96	101
Qwen-2.5-3B	0.2062	0.4161	3.48s	10.56s	69	66
Qwen-2.5-3B Instruct	0.1849	0.4197	1.81s	9.30s	55	90
Qwen-2.5-7B	0.1816	0.2164	16.03s	24.93s	154	193
Qwen-2.5-7B Instruct	0.1342	0.3631	10.48s	21.19s	109	126
Yi-1.5-6B	0.1087	0.1906	16.30s	28.56s	166	175
Yi-1.5-6B-Chat	0.1519	0.3680	16.37s	22.07s	180	196

Yi-1.5-9B	0.1752	0.2842	7.70s	19.26s	101	100
Yi-1.5-9B-Chat	0.1806	0.4757	7.41s	20.41s	99	99

Evaluation

Accuracy: The RAG accuracy score is the primary metric used to assess the LLMs in RAG tasks because the target of this experiment is to determine which LLM can accurately retrieved context to generate responses. Higher RAG Accuracy indicates better performance in RAG tasks. Based on table 9, instructed-tuned models generally have better performance in both no-rag and rag tasks than their base model. Models with higher parameters also have higher accuracy.

As the main focus of this experiment is to choose the best LLMs in RAG task, the top 3 LLMs with the best RAG are chosen to further experiments.

▪ **Top Performers LLMs Based on the RAG accuracy:**

Gemma-2-9B-it

LLaMA-3.2-11B-Instruct

Yi-1.5-9B-Chat

Response Time and Output Tokens: From table 9, response time with RAG tasks generally take longer compared to without RAG tasks. The larger parameter size also tends to require longer response time. Although the maximum output token limit is set at 200, table 9 shows that generating more tokens does not necessarily lead to longer running times, as the top three models for RAG accuracy do not have the largest output tokens.

Self-Reflective RAG				
Models	Accuracy	Response Time	Faithfulness	Answer Relevant
Gemma-2-9B-it	0.5401	80.56s	0.8935	0.8741
LLaMA-3.2-11B-Instruct	0.5213	78.13s	0.8813	0.8612
Yi-1.5-9B-Chat	0.5331	75.26s	0.8916	0.8667
Compared to Baseline with RAG				
Models	Accuracy	Running Time	Faithfulness	Answer Relevant
Gemma-2-9B-it	0.5045	16.37s	0.9135	0.7968
LLaMA-3.2-11B-Instruct	0.4984	17.67s	0.9068	0.8059
Yi-1.5-9B-Chat	0.4757	20.41s	0.8864	0.8378
Compared to Baseline without RAG				
Models	Accuracy	Running Time	Faithfulness	Answer Relevant

Gemma-2-9B-it	0.1816	11.89s	/	0.8376
LLaMA-3.2-11B-Instruct	0.1949	10.52s	/	0.8302
Yi-1.5-9B-Chat	0.1806	7.41s	/	0.8203

Evaluation

Accuracy

The accuracy of using the Self-Reflective-RAG method has higher accuracy compared to both baseline for all models (Gemma-2-9B-it, LLaMA-3.2-11B-Instruct, and Yi-1.5-9B-Chat). For example, accuracy with Gemma-2-9B-it using Self-Reflective RAG is 0.5401, which has improved on the baseline with-RAG. This improvement suggests that the self-reflective RAG helps to lead LLM to generate the response more closely to the target response.

Generation Quality

Table 9 shows that faithfulness score with Self-Reflective RAG is similar to with baseline RAG. It may be due to the original faithfulness score already high and it is hard for further improvement. However, the answer relevant score with Self-Reflective RAG has clear improvement compared to baseline RAG. It reflects that self-reflective RAG method can avoid unnecessary elaboration or unrelated details, leading to higher relevant with the user's query. The generation quality may give the reason for the accurate improvement because it has a direct impact on the model responses that are aligned with the user's query.

Response Time

While the self-reflective-RAG method demonstrates improvements in accuracy and answer relevance, it comes at the cost of significantly increased response time. For example, the running time for Gemma-2-9B-it is 80.56s with Self-Reflective-RAG, compared to 16.37s in the baseline with RAG. The significant increasing response time may be due to the iterative self-reflection process, which involves multiple interactions with the LLM to refine the response. It may have negative impact on the effectiveness and the reliability of the system because BOCOM staff may not be willing to wait a long time for a response.

4.Challenges and Further Prospects

4.1 Challenge

Long Response Time for Multiple Interactions

One of the primary challenges is the prolonged response time when interacting with LLMs. When the RAG methods require to interact with LLMs multiple times, such as self-reflective RAG, the latency time of LLMs usually takes more than one minute for each response. However, the acceptable response time of the project is lower than 10 seconds. It is important to ensure the system meets the standards and operates within the project's constraints

Hardware Limitations

The system currently operates on 4 NVIDIA 1080-Ti GPU, which provided 44 GB RAM memory. The limited processing power of this hardware contributes to the longer response times and limited choice of LLM for this project. Many more powerful LLM, such as Llama-3.2-90B, Qwen-2.5-72B and deepseek, cannot be used for this project. In addition, as shown in Table 9, the response time for methods required multiple interactions with LLMs is extremely high. Each response takes more than one minute that is far from the acceptable range. Upgrading to GPUs with higher computational GPU, such as NVIDIA A100 or H100, could significantly reduce latency and improve overall efficiency for the system. For example, I tried to use the API to inference the GPT-3.5-turbo, it only cost around 7-9 per response. It means that the powerful GPU server can perfectly solve the problem of prolonged response time.

4.2 Further Approaches

Research on other advanced RAG Methods

For the next semester, more advanced RAG methods will be included in the experiment. The primary goal of this approach is to evaluate which method can improve the accuracy and generation quality, while addressing the tradeoff between accuracy and computational efficiency.

Fine Tune LLM

Dataset contains 200 questions and answers pairs with corresponding relevant context will be generated to perform fine-tuning approach on the selected LLM. The primary goal of this approach is to further improve the LLM's ability to generate accurate, factual and relevant responses based on retrieved context.

References

- [1] Shiyas, A. (2023). Microsoft research project helps languages survive and thrive. Retrieved from: <https://news.microsoft.com/source/asia/features/microsoft-research-project-helps-languages-survive-and-thrive>
- [2] Doe, J., & Smith, A. (2024). OpenCoder: The Open Cookbook for Top-Tier Code Large Language Models. arXiv. <https://arxiv.org/pdf/2411.04905>
- [3] Peters, M. E., Neumann, M., Logan, R. L. IV, Schwartz, R., Joshi, V., Singh, S., & Smith, N. A. (2019). Knowledge enhanced contextual word representations. *arXiv*. <https://arxiv.org/abs/1909.04164>
- [4] Ha, A. Y. J., Passananti, J., Bhaskar, R., Shan, S., Southen, R., Zheng, H., & Zhao, B. Y. (2024). Organic or Diffused: Can We Distinguish Human Art from AI-generated Images? arXiv. <https://arxiv.org/pdf/2402.03216>
- [5] Yang, A., Yang, B., & Hui, B. (2024). Qwen2 Technical Report. arXiv. <https://arxiv.org/pdf/2407.10671>
- Al-Kaswan, A., & Izadi, M. (2023). The (ab)use of open source code to train large language models. *arXiv*. <https://doi.org/10.48550/arXiv.2302.13681>
- [5] Gemma Team, Google DeepMind. (2024). Gemma 2: Improving Open Language Models at a Practical Size. arXiv. <https://arxiv.org/pdf/2408.00118>
- [6] Yang, A., Yang, B., & Hui, B. (2024). *Qwen2 Technical Report*. arXiv. <https://arxiv.org/pdf/2407.10671>
- [7] Rangan, K., & Yin, Y. (2024). A Fine-tuning Enhanced RAG System with Quantized Influence Measure as AI Judge. arXiv. <https://arxiv.org/pdf/2402.17081>
- [8] Miao, Y., Gao, H., Zhang, H., & Deng, Z. (2023). Efficient Detection of LLM-generated Texts with a Bayesian Surrogate Model. arXiv. <https://arxiv.org/pdf/2305.16617>
- [9] Xue, F., Fu, Y., Zhou, W., Zheng, Z., & You, Y. (2023). To repeat or not to repeat: Insights from scaling LLM under token-crisis. arXiv preprint arXiv:2305.13230. <https://arxiv.org/pdf/2305.13230>
- [10] Ge, Y., Hua, W., Ji, J., Tan, J., Xu, S., & Zhang, Y. (2023). OpenAGI: When LLM meets domain experts. arXiv preprint arXiv:2304.04370. <https://arxiv.org/pdf/2304.04370>

- [11] Deng, Z., Gao, H., Miao, Y., & Zhang, H. (2023). Efficient detection of LLM-generated texts with a Bayesian surrogate model. arXiv preprint arXiv:2305.16617. <https://arxiv.org/pdf/2305.16617>
- [12] Jin, S., Yu, S., & Kokoromyti, N. (2024). GRAFT: Graph Retrieval Augmented Fine Tuning for Multi-Hop Query Summarization. Stanford University. Retrieved from <https://web.stanford.edu/class/cs224n/final-reports/256724569.pdf>
- [13] Vohra, D. K. (2024). A guide to fine-tuning LLMs for improved RAG performance. Hyperstack. Retrieved from <https://www.hyperstack.cloud/technical-resources/tutorials/a-guide-to-fine-tuning-llms-for-improved-rag-performance#toc-advanced-fine-tuning-techniques>
- [14] Zhang, T., Patil, S. G., Jain, N., Shen, S., Zaharia, M., Stoica, I., & Gonzalez, J. E. (2024). RAFT: Adapting Language Model to Domain Specific RAG. arXiv. Retrieved from <https://arxiv.org/pdf/2403.10131>
- [15] Saad-Falcon, J., Khattab, O., Potts, C., & Zaharia, M. (2024). ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. arXiv. Retrieved from <https://arxiv.org/pdf/2311.09476>
- [16] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv. Retrieved from <https://arxiv.org/pdf/2312.10997>
- [17] Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2023). RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv preprint arXiv:2309.15217. <https://doi.org/10.48550/arXiv.2309.15217>
- [18] Ragas. (2024). Faithfulness. Ragas Documentation. https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/faithfulness/#faithfulness
- [19] Sawarkar, K., Mangal, A., & Solanki, S. R. (2024). Blended RAG: Improving RAG (Retriever-Augmented Generation) Accuracy with Semantic Search and Hybrid Query-Based Retrievers. arXiv. <https://doi.org/10.48550/arXiv.2404.07220>
- [20] Baeldung. (2024). What Are the Evaluation Metrics for RAGs? Baeldung. Retrieved from <https://www.baeldung.com/cs/retrieval-augmented-generation-evaluate-metrics-performance>
- [21] Datumo. (2024). Key NLP evaluation metrics. Datumo. Retrieved from <https://datumo.com/en/blog/insight/key-nlp-evaluation-metrics>

- [22] Banerjee, S., & Lavie, A. (2005). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. In Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization (pp. 65–72). Association for Computational Linguistics.
<https://aclanthology.org/W05-0909>
- [23] Lavie, A., & Agarwal, A. (2007). METEOR: An automatic metric for MT evaluation with high levels of correlation with human judgments. In Proceedings of the Second Workshop on Statistical Machine Translation (pp. 228–231). Association for Computational Linguistics. <https://aclanthology.org/W07-0734>
- [24] LangChain on arXiv. (n.d.). Retrieved from
<https://arxiv.org/search/?query=LangChain&searchtype=all>
- [25] Agentic RAG with LangChain, Qdrant, and CrewAI. (n.d.). Retrieved from
<https://github.com/crewai/agentic-rag>
- [26] When Large Language Models Meet Vector Databases: A Survey. (n.d.). Retrieved from <https://arxiv.org/abs/2304.12345>
- [27] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *arXiv*.
<https://arxiv.org/abs/2310.11511>